IMAGINE (Image generative AI) Dataset - AI generated Images

Ujas Bhuva
40262555
ujasbhuva@gmail.com

Mihir Gediya
40272399
mihirgediya2001@gmail.com

Arik Kantesaria
40272574
arikkantesaria9@gmail.com

Pranavkumar Sakariya
40264062
pranavsakariya1309@gmail.com

Yash Chhelaiya
40257682
yash.chhelaiya@gmail.com

ABSTRACT

IMAGINE is a cutting-edge Image browsing and Generation web application. Leveraging the power of AWS OpenSearch, IMAGINE offers a fast and intuitive interface for prompt-based image searches and processing queries in a fraction of a time. This innovative app harnesses the scalability and reliability of a Kubernetes cluster on the Google Cloud Platform (GCP) to effortlessly manage the dynamic load of the hosted AI Image generation model, SDXL. To ensure uninterrupted service, IMAGINE employs AWS services such as SQS, Lambda, and CloudWatch to execute image requests efficiently, maintaining optimal performance and responsiveness. The application's backend is fortified with an S3 bucket, ensuring secure and scalable storage for the generated images. To enhance data management, IMAGINE seamlessly integrates with a hosted Neon DB, providing a robust solution for storing and retrieving essential information associated with the generated images. The synergy of these technologies empowers IMAGINE to deliver a seamless and rapid image browsing experience. It establishes a scalable and resilient architecture that uses Kubernetes to ensure the app's performance even under heavy loads. IMAGINE is a testament to the convergence of cutting-edge cloud services and advanced image processing models, promising users an unparalleled journey into instant image gratification.

1 INTRODUCTION

IMAGINE introduces a robust, full-stack application utilizing Python and NextJS to support the era of artificial intelligence. This application's main point is deploying the image-generative model SDXL, which is integrated into a distributed system. Using a Kubernetes cluster within the Google Cloud Platform (GCP) ensures efficient load distribution for image generation, optimizing server performance. This app uses AWS SQS and AWS Lambda to avoid potential bottlenecks, facilitating minimum connection times between the front and back-end. The application provides the feature of an image browsing functionality using OpenSearch indexes to explore a vast 2.4 GB dataset. Notably, a hybrid infrastructure incorporating both AWS and GCP services is used to optimize hosting costs.

2 SYSTEM

In this section, we will discuss the different components of IMAGINE:

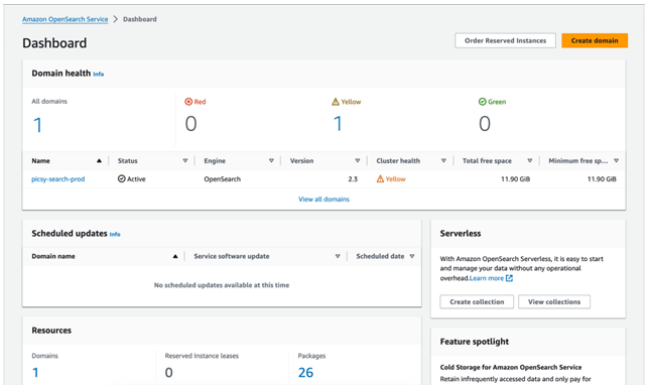


Figure 1: Hosted OpenSearch index of dataset

2.1 Dataset

A dataset of ai-generated images is a dataset that contains links to images and prompts along with the other data. It contains multiple image links for a single row and contains 2.4 million rows in total.

2.2 OpenSearch for indexing dataset

AWS provides OpenSearch to index the data and uses an ElasticSearch-like approach to make searches faster. For hosting the OpenSearch index, AWS uses an ec2 instance to process the incoming query and OpenSearch-based storage to store the indexes. Users can enter the query from the front-end to get AI-generated Images. The backend will process that query, make a searchable query, and execute that query on OpenSearch. Figure 1 shows the OpenSearch console, which currently has one index.

2.3 Dockerized FastAPI

This application uses a Python-based FastAPI framework to handle the API request and to communicate between the front-end and the database. Currently, this backend service is dockerized. It also uses the Google Artifact Registry to save the built images. Figure 2 shows the code of the Dockerfile; Figure 3 illustrates porter_stack_backend.yml to run the deployment pipeline in Kubernetes.

2.4 Kubernetes cluster on GCP

We are using the Kubernetes cluster to host the fast API backend for this application. Currently, there are a minimum of one and

Dockerfile

You, 16 hours ago | 1 author (You)

```
FROM python:3.10

WORKDIR /app

COPY . /app

RUN apt-get update && apt-get install ffmpeg libsm6 libxext6  -y
RUN pip install wheel
RUN pip install -r requirements.txt


EXPOSE 80

CMD ["uvicorn", "server.app:app", "--host", "0.0.0.0", "--port", "80"]
```

Figure 2: Code of Dockerfile

.github > workflows > ! porter_stack_backend.yml

You, 15 hours ago | 1 author (You)

```
version: v2
name: backend
services:
- name: fastapi
  run: ""
  type: web
  instances: 1
  cpuCores: 3.5
  ramMegabytes: 15450
  terminationGracePeriodSeconds: 30
  port: 80
  autoscaling:
    enabled: true
    minInstances: 1
    maxInstances: 4
    cpuThresholdPercent: 80
    memoryThresholdPercent: 80
  ingressAnnotations:
    nginx.ingress.kubernetes.io/proxy-connect-timeout: '"120"'
    nginx.ingress.kubernetes.io/proxy-read-timeout: '"120"'
    nginx.ingress.kubernetes.io/proxy-send-timeout: '"120"'
build:
  context: ./
  method: docker
  dockerfile: ./Dockerfile
env:
  env_file: .env      You, 15 hours ago • Uncommitted changes
```

Figure 3: Code for Deployment pipeline

a maximum of six computing nodes in the cluster to handle the image generation load. We have configured an 80% threshold for memory and CPU usage to establish a new cluster node if the system exceeds that range. We are using the us-east1 zone of GCP for this app to deploy the nodes with 32 GB RAM and eight vCPUs in each node. We use a third-party service named Porter to deploy apps using automated deployment jobs and get an SSL-based domain. While deploying each build, CI/CD will generate a new image of the application and then push it to the Google Artifact Registry. Then, porter.run will deploy the generated image to the cluster.

2.5 AWS SQS for queuing image generation Request

- AWS SQS provides a system to manage message queuing and add triggers based on events happening in the Queue.
- We use a standard FIFO queue "image-generation-obj" deployed in AWS to manage the image generation requests.

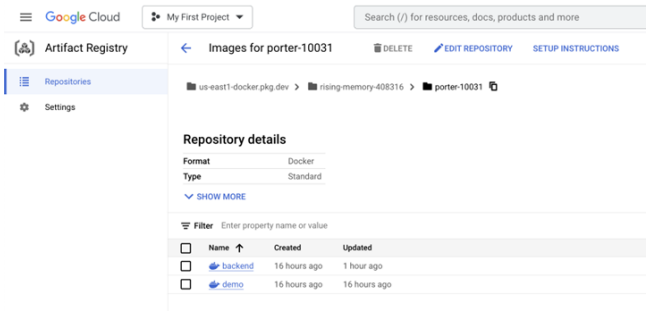


Figure 4: Docker Artifact registry on GCP

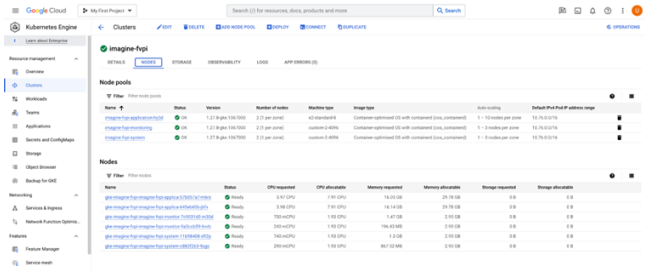


Figure 5: Kubernetes cluster

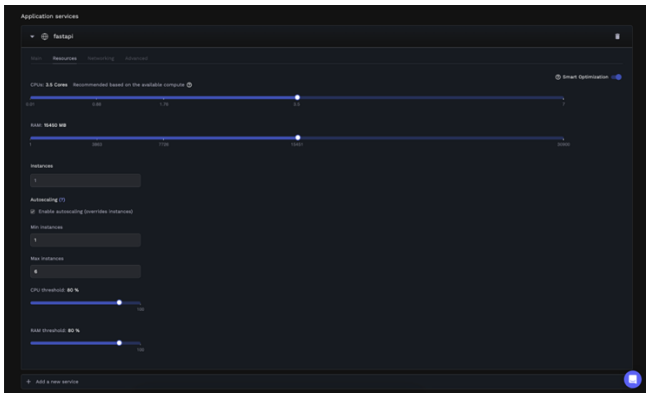


Figure 6: Configuration on porter for nodes in cluster and threshold

- This mechanism will reduce the sudden load on the whole system and the connection time with the backend for generating images. Also, a single user can add multiple requests without waiting for one to finish.
- This SQS queue is connected with the lambda function "image-request-executor." SQS will trigger a lambda function with the message appended in the Queue, and lambda will execute the further instruction defined in the Function.
- We have added a window for triggering the Function with five messages and 5 seconds. That means SQS will wait 5 seconds or five messages before triggering the lambda function.
- Figure 7 shows the Queue deployed in the AWS console.

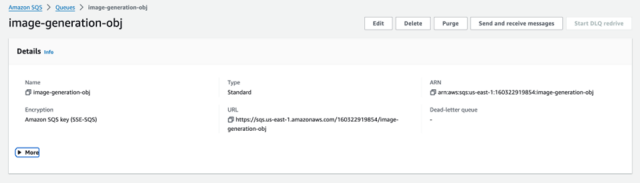


Figure 7: Deployed SQS on in AWS console

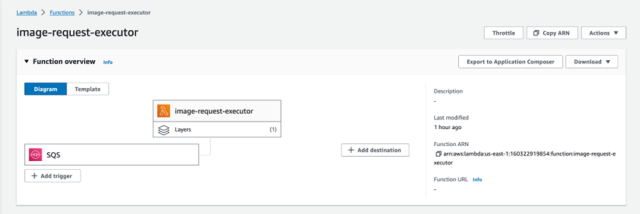


Figure 8: Lamda function deployed in AWS

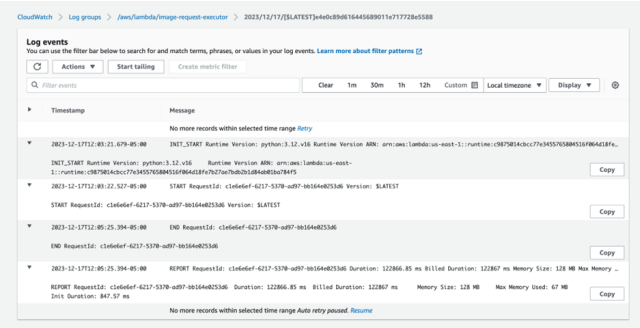


Figure 9: Logs of executed Lambda function from SQS

2.6 AWS Lambda for making requests to the backend

- AWS Lambda provides a system to deploy a serverless function that can be invoked based on events happening in the services defined or deployed in an AWS account.
- We use a Python-based lambda function to request the backend deployed in the Kubernetes cluster.
- Here, the lambda function will consume the SQS message and call the backend. A lambda function can wait for a maximum of 15; we are assuming that the backend is capable of making execution within this time (even in case it needs to deploy a new node in a cluster).
- Figure 8 shows a lambda function deployed in the AWS.

2.7 AWS CloudWatch

AWS CloudWatch provides a system to manage the logging of different services in AWS. For this application, we use CloudWatch to maintain the logs of the Lambda function's execution. Figure 9 shows the logs of the executed Lambda function from SQS.

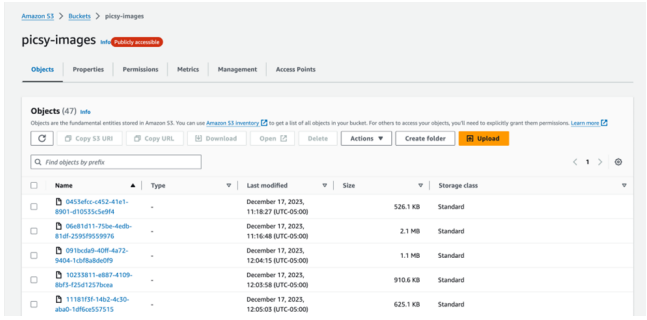


Figure 10: established S3 bucket on AWS.

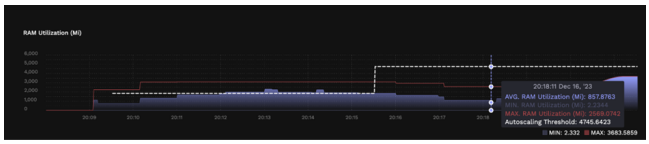


Figure 11: Memory usage

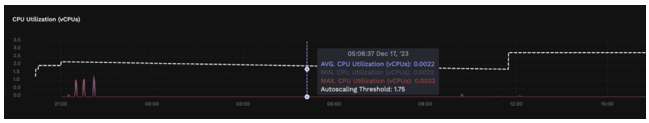


Figure 12: CPU usage

2.8 AWS S3 Bucket

AWS S3 Bucket is a platform for managing the content delivery network. We can host different file objects in bucket space. This service is used here to host the generated Images. Figure 10 shows the established S3 bucket on AWS.

3 DEMO OF APPLICATION

3.1 Horizontal scaling

For testing the horizontal scaling, we pushed many requests to the cluster via SQS and tried to hit the threshold value that would deploy the new node in the cluster. We noted the RAM usage from Porter. Here, as the Figure 11 shows, during the time 20:11, the application hit the threshold, and the application added a new node in the cluster, so after a few minutes at 20:15, it deployed a new node, and the threshold memory became higher. This same applies to CPU usage, shown in Figure 12.

3.2 Fault tolerance

- By killing one of the nodes in the cluster, we can continue generating new images using the model.
- One more mechanism is added; we can forcefully fail requests by selecting force fail from the web app. This flag will raise an error from the backend, and the lambda function will add that message back to SQS with an increased fail_count by 1. It will try to execute that request message a maximum of three times. If it is still failing, it will not

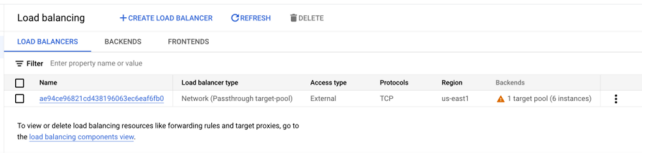


Figure 13: Load balancer pool

execute the message and fail afterward (we can consider this scenario as failure due to a programming error).

3.3 Load Balancer

In this system, we can define one load balancer instance with any configuration of RAM and vCPU units. In GCP, a load balancer node is defined for a target pool of 6 instances.

4 CONCLUSION

We can build robust, fault-tolerant, and scalable systems using the distributed system for generating AI images.

(1) **Performance:** We can improve the application's overall performance using many nodes in the cluster because many nodes can handle many requests. Also, by using SQS, we can reduce the waiting time for users to send requests for generating multiple images.
(2) **Scalability:** Using the Kubernetes cluster, we can distribute a load of the application among many nodes and make it dynamic to deploy new nodes based on load.
(3) **Fault Tolerant:** Horizontal scaling can help handle faults in the app.

REFERENCES

(1) AWS OpenSearch documentation
(2) AWS SQS API documentation
(3) AWS Lambda documentation
(4) Porter.run application documentation for GCP
(5) Load balancer in kubernetes in GCP
(6) AWS S3 bucket documentation
(7) Vercel documentation
(8) Ai-generated images kaggle dataset